llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5_exact · claude-haiku-4-5-20251001 · FASTER

−87 / +297 lines of 1967 · 21 parallel region(s) added · 29 of 87 candidate(s) kept · RESTRUCTURING (224 code line(s) added, 80 removed) · restructuring level 0

BEFORE

```
308
309       Real_t fx_local[8] ;
310       Real_t fy_local[8] ;
311       Real_t fz_local[8] ;
312
313     for( Index_t k=0 ; k<numElem ; ++k )
314     {
315       const Index_t* const elemToNode = domain.nodelist(k);
316       Real_t B[3][8] ;
317       Real_t x_local[8] ;
    …
328
329       SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
330                                    fx_local, fy_local, fz_local ) ;
331
332       for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
333          Index_t gnode = elemToNode[lnode];
334          domain.fx(gnode) += fx_local[lnode];
335          domain.fy(gnode) += fy_local[lnode];
336          domain.fz(gnode) += fz_local[lnode];
337       }
338
339     }
340
341   }
342
343   static inline
344   void VoluDer(const Real_t x0, const Real_t x1, const Real_t x2,
    …
376                                Real_t dvdz[8],
377                                const Real_t x[8],
378                                const Real_t y[8],
379                                const Real_t z[8])
380   {
381      VoluDer(x[1], x[2], x[3], x[4], x[5], x[7],
382              y[1], y[2], y[3], y[4], y[5], y[7],
383              z[1], z[2], z[3], z[4], z[5], z[7],
384              &dvdx[0], &dvdy[0], &dvdz[0]);
385      VoluDer(x[0], x[1], x[2], x[7], x[4], x[6],
386              y[0], y[1], y[2], y[7], y[4], y[6],
387              z[0], z[1], z[2], z[7], z[4], z[6],
388              &dvdx[3], &dvdy[3], &dvdz[3]);
389      VoluDer(x[3], x[0], x[1], x[6], x[7], x[5],
390              y[3], y[0], y[1], y[6], y[7], y[5],
391              z[3], z[0], z[1], z[6], z[7], z[5],
392              &dvdx[2], &dvdy[2], &dvdz[2]);
393      VoluDer(x[2], x[3], x[0], x[5], x[6], x[4],
394              y[2], y[3], y[0], y[5], y[6], y[4],
395              z[2], z[3], z[0], z[5], z[6], z[4],
396              &dvdx[1], &dvdy[1], &dvdz[1]);
397      VoluDer(x[7], x[6], x[5], x[0], x[3], x[1],
398              y[7], y[6], y[5], y[0], y[3], y[1],
399              z[7], z[6], z[5], z[0], z[3], z[1],
400              &dvdx[4], &dvdy[4], &dvdz[4]);
401      VoluDer(x[4], x[7], x[6], x[1], x[0], x[2],
402              y[4], y[7], y[6], y[1], y[0], y[2],
403              z[4], z[7], z[6], z[1], z[0], z[2],
404              &dvdx[5], &dvdy[5], &dvdz[5]);
405      VoluDer(x[5], x[4], x[7], x[2], x[1], x[3],
406              y[5], y[4], y[7], y[2], y[1], y[3],
407              z[5], z[4], z[7], z[2], z[1], z[3],
408              &dvdx[6], &dvdy[6], &dvdz[6]);
409      VoluDer(x[6], x[5], x[4], x[3], x[2], x[0],
410              y[6], y[5], y[4], y[3], y[2], y[0],
411              z[6], z[5], z[4], z[3], z[2], z[0],
412              &dvdx[7], &dvdy[7], &dvdz[7]);
```

AFTER

```
308
309       Real_t fx_local[8] ;
310       Real_t fy_local[8] ;
311       Real_t fz_local[8] ;
312
313       Real_t *fx_all = Allocate<Real_t>(numElem * 8);
314       Real_t *fy_all = Allocate<Real_t>(numElem * 8);
315       Real_t *fz_all = Allocate<Real_t>(numElem * 8);
316
317     #pragma omp parallel for shared(domain, sigxx, sigyy, sigzz, determ, fx_all,
318         firstprivate(fx_local, fy_local, fz_local)
319     for( Index_t k=0 ; k<numElem ; ++k )
320     {
321       const Index_t* const elemToNode = domain.nodelist(k);
322       Real_t B[3][8] ;
323       Real_t x_local[8] ;
    …
334
335       SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
336                                    fx_local, fy_local, fz_local ) ;
337
338       for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
339          fx_all[k*8+lnode] = fx_local[lnode];
340          fy_all[k*8+lnode] = fy_local[lnode];
341          fz_all[k*8+lnode] = fz_local[lnode];
342       }
343
344     }
345
346     for( Index_t k=0 ; k<numElem ; ++k ) {
347       const Index_t* const elemToNode = domain.nodelist(k);
348       for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
349          Index_t gnode = elemToNode[lnode];
350          domain.fx(gnode) += fx_all[k*8+lnode];
351          domain.fy(gnode) += fy_all[k*8+lnode];
352          domain.fz(gnode) += fz_all[k*8+lnode];
353       }
354     }
355
356     Release(&fx_all);
357     Release(&fy_all);
358     Release(&fz_all);
359
360   }
361
362   static inline
363   void VoluDer(const Real_t x0, const Real_t x1, const Real_t x2,
    …
395                                Real_t dvdz[8],
396                                const Real_t x[8],
397                                const Real_t y[8],
398                                const Real_t z[8])
399   {
400      /* Eight independent VoluDer calls computing volume derivatives for each
401         element face. Restructured as a loop to expose parallelism: each iterati
402         writes to a unique output element with no data dependences between itera
403     #pragma omp parallel for shared(x, y, z, dvdx, dvdy, dvdz)
404     for (Index_t i = 0; i < 8; ++i) {
405        switch(i) {
406           case 0:
407              VoluDer(x[1], x[2], x[3], x[4], x[5], x[7],
408                      y[1], y[2], y[3], y[4], y[5], y[7],
409                      z[1], z[2], z[3], z[4], z[5], z[7],
410                      &dvdx[0], &dvdy[0], &dvdz[0]);
411              break;
412           case 1:
413              VoluDer(x[2], x[3], x[0], x[5], x[6], x[4],
414                      y[2], y[3], y[0], y[5], y[6], y[4],
415                      z[2], z[3], z[0], z[5], z[6], z[4],
416                      &dvdx[1], &dvdy[1], &dvdz[1]);
417              break;
418           case 2:
419              VoluDer(x[3], x[0], x[1], x[6], x[7], x[5],
420                      y[3], y[0], y[1], y[6], y[7], y[5],
421                      z[3], z[0], z[1], z[6], z[7], z[5],
422                      &dvdx[2], &dvdy[2], &dvdz[2]);
423              break;
424           case 3:
425              VoluDer(x[0], x[1], x[2], x[7], x[4], x[6],
426                      y[0], y[1], y[2], y[7], y[4], y[6],
427                      z[0], z[1], z[2], z[7], z[4], z[6],
428                      &dvdx[3], &dvdy[3], &dvdz[3]);
429              break;
430           case 4:
431              VoluDer(x[7], x[6], x[5], x[0], x[3], x[1],
432                      y[7], y[6], y[5], y[0], y[3], y[1],
433                      z[7], z[6], z[5], z[0], z[3], z[1],
434                      &dvdx[4], &dvdy[4], &dvdz[4]);
435              break;
436           case 5:
437              VoluDer(x[4], x[7], x[6], x[1], x[0], x[2],
438                      y[4], y[7], y[6], y[1], y[0], y[2],
439                      z[4], z[7], z[6], z[1], z[0], z[2],
440                      &dvdx[5], &dvdy[5], &dvdz[5]);
```

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is